4DT906 Assignment 1: Parallel Computing
Odd-Even Sort and Pearson Correlation Coefficient (C/C++)

Samuel Berg, Jesper Wingren, Emil Ulvagården

March 9, 2026

Abstract

This report presents parallel implementations of two algorithms: the odd-even transposition sort and the Pearson correlation coefficient (PCC). Both algorithms are implemented using C++ `std::thread` for shared-memory parallelism. Correctness is validated against the provided sequential reference implementations.

Contents

1 Introduction **2**

2 Benchmarking Setup **2**

3 Odd-Even Transposition Sort **2**
3.1 Algorithm Description . . . 2
3.2 Sequential Implementation . . . 2
3.3 Parallel Implementation — `std::thread` (`oddevensort_par.cpp`) . . . 3
3.3.1 Parallelisation Strategy . . . 3
3.3.2 Synchronisation . . . 3
3.3.3 Data Decomposition . . . 3
3.4 Correctness Validation . . . 3
3.5 Performance Results — Odd-Even Sort . . . 3

4 Pearson Correlation Coefficient **4**
4.1 Algorithm Description . . . 4
4.2 Sequential Implementation . . . 4
4.3 Parallel Implementation — `std::thread` (`pcc_par.cpp`) . . . 4
4.3.1 Parallelisation Strategy . . . 4
4.3.2 Data Decomposition . . . 4
4.3.3 Synchronisation . . . 5
4.4 Correctness Validation . . . 5
4.5 Performance Results — Pearson Correlation Coefficient . . . 5

5 Discussion **5**
5.1 Scalability and Observed Behaviour . . . 5
5.2 Memory Behaviour . . . 6
5.3 Load Balancing . . . 6

6 Conclusion **6**

1 Introduction

Parallel computing enables large-scale computations to complete in less time by distributing work across multiple execution units. This assignment explores two algorithms that are amenable to parallelisation:

1. **Odd-Even Transposition Sort**: A comparison-based sorting algorithm with a regular, phase-alternating structure where independent pairs can be processed in parallel.
2. **Pearson Correlation Coefficient (PCC)**: A statistical measure of linear correlation between pairs of data rows. Computing all $\frac{m(m-1)}{2}$ pair-wise correlations for an $m \times n$ matrix is computationally intensive and highly parallelisable.

For each algorithm a correct sequential reference implementation is provided (`oddevensort.cpp` and `pcc_seq.cpp`). Parallel versions were developed on top of these references without modifying them.

2 Benchmarking Setup

- **Hardware**: **CPU: AMD Ryzen 9 5950X, 16-Cores, 32GB RAM**
- **Operating System**: **WSL Ubuntu 24.04.4 LTS and Kernal 6.6.87.2-microsoft-standard-WSL2**
- **Compiler**: g++ with `-O2` optimisation, C++17 standard
- **Thread counts tested**: **32 threads**
- **Number of runs per configuration**: **5 runs**

3 Odd-Even Transposition Sort

3.1 Algorithm Description

The odd-even transposition sort alternates between two phases until the array is sorted:

- **Even phase**: compare-and-swap all pairs $(v[0], v[1])$, $(v[2], v[3])$, ...
- **Odd phase**: compare-and-swap all pairs $(v[1], v[2])$, $(v[3], v[4])$, ...

After n full iterations (even + odd) the array is guaranteed to be sorted. Within each phase all active pairs are independent, making the algorithm naturally parallel.

3.2 Sequential Implementation

The reference implementation (`oddevensort.cpp`) performs n outer iterations and processes each active pair in the inner loop:

```
for (int i = 1; i <= s; i++) {
    for (int j = i % 2; j < s-1; j = j + 2) {
        if (numbers[j] > numbers[j + 1]) {
            std::swap(numbers[j], numbers[j + 1]);
        }
    }
}
```

Listing 1: Sequential Odd-Even Sort (core loop)

Time complexity: $O(n^2)$ in the worst case.

3.3 Parallel Implementation — `std::thread` (`oddevensort_par.cpp`)

3.3.1 Parallelisation Strategy

Within each phase the $\lfloor n/2 \rfloor$ active compare-swap pairs are independent. The pairs are divided into contiguous blocks, one block per thread, so threads operate on disjoint memory locations and no data races occur. A reusable spin-wait barrier synchronises all threads between phases, ensuring phase $k+1$ does not begin before phase k has finished.

3.3.2 Synchronisation

A custom `Barrier` class based on `std::atomic<int>` provides the inter-phase synchronisation. Each thread calls `arrive_and_wait()` at the end of its phase work; the last arriving thread resets the counter and increments a generation counter, waking the others.

3.3.3 Data Decomposition

Pairs are divided into contiguous blocks to maximise cache locality. Thread t handles pairs `[pair_start, pair_end)` where the block boundaries are computed to distribute the remainder evenly:

```
int per        = num_pairs / num_threads;
int rem        = num_pairs % num_threads;
int pair_start = thread_id * per + (thread_id < rem ? thread_id : rem);
int pair_end   = pair_start + per + (thread_id < rem ? 1 : 0);
```

Listing 2: Contiguous pair-block assignment

3.4 Correctness Validation

The parallel implementation is verified with `std::is_sorted` after execution. The default input size is $2^{19} = 524\,288$ elements; additional sizes were also tested.

3.5 Performance Results — Odd-Even Sort

Input size	Threads	Sequential (s)	Parallel (s)	Speedup
100 000	32	3.94825	0.749985	5.26x
2^{19}	32	131.659	14.073	9.35x
100 000	32	3.99279	0.514602	7.75x
2^{19}	32	132.319	10.3626	12.76x
100 000	32	3.94861	0.556298	7.09x
2^{19}	32	131.531	13.2148	9.95x
100 000	32	3.99469	0.719911	5.54x
2^{19}	32	133.462	13.5309	9.86x
100 000	32	3.73586	0.688854	5.42x
2^{19}	32	132.198	11.6074	11.38x

Table 1: Odd-Even Sort — `std::thread` implementation

4 Pearson Correlation Coefficient

4.1 Algorithm Description

The Pearson correlation coefficient between two vectors x and y of length n is:

$$r = \frac{\sum_{k=1}^{n}(x_k - \bar{x})(y_k - \bar{y})}{\sqrt{\sum_k (x_k - \bar{x})^2}\,\sqrt{\sum_k (y_k - \bar{y})^2}}$$

For a matrix with m rows and n columns, all $\frac{m(m-1)}{2}$ row-pair correlations are computed. The calculation proceeds in three steps:

1. Compute the row mean $\bar{x}_i$ for every row i.
2. Compute the mean-subtracted matrix $(x_{ik} - \bar{x}_i)$ and the row standard deviation σ_i.
3. Compute each pair-wise correlation $r_{ij} = \frac{\sum_k (x_{ik}-\bar{x}_i)(x_{jk}-\bar{x}_j)}{\sigma_i \sigma_j}$.

4.2 Sequential Implementation

The reference implementation (`pcc_seq.cpp`) computes all three steps in sequence. The correlation loop is:

```
for (sample1 = 0; sample1 < ROWS-1; sample1++) {
    for (sample2 = sample1+1; sample2 < ROWS; sample2++) {
        sum = 0.0;
        for (i = 0; i < COLS; i++) {
            sum += mm[sample1*COLS+i] * mm[sample2*COLS+i];
        }
        r = sum / (std[sample1] * std[sample2]);
        output[...] = r;
    }
}
```

Listing 3: Sequential PCC correlation loop

Time complexity: $O(m^2 n)$.

4.3 Parallel Implementation — `std::thread` (`pcc_par.cpp`)

4.3.1 Parallelisation Strategy

All three steps (mean, deviation/std, correlations) are parallelised. Steps 1 and 2 decompose the work by rows: each thread receives a contiguous slice of rows. Step 3 uses interleaved (round-robin) assignment of `sample1` rows across threads, which distributes the triangular workload evenly because rows with a higher index produce more pairs.

4.3.2 Data Decomposition

- **Steps 1 & 2 (linear)**: rows $[0, m)$ are split into contiguous blocks of size $\lceil m/T \rceil$ using a helper lambda.
- **Step 3 (triangular)**: thread t processes `sample1` rows t, $t+T$, $t+2T$, ... (stride T). Because each output index `sample1*ROWS + sample2 - offset` is written by exactly one thread, no synchronisation is required inside the correlation loop.

4.3.3 Synchronisation

No explicit synchronisation is needed within steps 1–3 because the work partitions are non-overlapping. Thread joining (`th.join()`) serves as the implicit barrier between steps.

4.4 Correctness Validation

The provided `verify` program compares the floating-point output files of the sequential and parallel implementations. The parallel version is validated for all tested matrix sizes.

4.5 Performance Results — Pearson Correlation Coefficient

The table entries are computed by

$$Mean = \frac{\sum values}{N}$$

where N is the number of runs and $Mean$ is the reported values.

Matrix size	Threads	Sequential (s)	Parallel (s)	Speedup
64×64	32	0.00010	0.00472	0.020x
128×128	32	0.00062	0.00468	0.132x
256×256	32	0.00496	0.00482	1.028x
512×512	32	0.04012	0.00788	5.094x
1024×1024	32	0.32336	0.03034	10.660x
2048×2048	32	2.67114	0.18982	14.090x
4096×4096	32	21.57692	1.47066	14.682x

Table 2: PCC — `std::thread` implementation

5 Discussion

5.1 Scalability and Observed Behaviour

The benchmarks reveal distinct scalability profiles for the two algorithms.

Odd-Even Sort achieves a mean speedup of approximately 6.2× for $n = 100\,000$ and 10.7× for $n = 2^{19}$ using 32 threads (parallel efficiency $\approx 19\%$ and 33% respectively). The speedup improves with larger inputs because each phase does more useful work relative to the fixed cost of the barrier synchronisation. However, the algorithm requires n barrier crossings in total, so synchronisation overhead scales with problem size at the same rate as the computation itself, capping efficiency well below the theoretical 32×. Run-to-run variability is also noticeable (5.3×–7.8× for $n = 100\,000$; 9.4×–12.8× for $n = 2^{19}$), likely caused by OS scheduling jitter and cache state differences between successive runs.

PCC shows a sharp crossover at a matrix size of approximately 256×256, where the parallel and sequential runtimes are nearly identical (1.03× speedup). Below this threshold, thread-spawn overhead, memory setup, and join latency dominate the very short computation, leading to a severe slowdown (only 0.02× at 64×64, i.e. the parallel version is $\approx 50\times$ slower). Above the crossover the computation grows as $O(m^2 n)$ and the speedup rises rapidly, reaching 10.7× at 1024×1024, 14.1× at 2048×2048, and 14.7× at 4096×4096 (parallel efficiency $\approx 46\%$). The levelling-off between 2048 and 4096 suggests that memory-bandwidth saturation is beginning to limit further gains.

A somewhat surprising result is that the odd-even sort achieves a comparable peak speedup ($\approx 10.7\times$) to PCC at large sizes, despite requiring $n \approx 500\,000$ barrier synchronisations. The

reason is that each synchronisation is extremely lightweight (spin-wait on a shared atomic counter) and the per-phase computation—even with 32 threads handling $\approx 8\,000$ compare-swap pairs each—is itself substantial enough to amortise the barrier cost.

- **Odd-Even Sort**: The algorithm performs n phases with a barrier between each phase. Synchronisation overhead grows with thread count, potentially limiting scalability for small arrays.
- **PCC**: The correlation step dominates runtime for large matrices, making PCC well-suited for parallelisation. The implementation is expected to scale well for large matrices where computation time dominates overhead.
- **Thread overhead vs. Computation**: For small inputs, thread management overhead can exceed the computational savings, leading to reduced speedup. As problem size increases, the compute-to-overhead ratio improves.

5.2 Memory Behaviour

- **Odd-Even Sort**: Contiguous pair-block assignment improves spatial locality. Each thread accesses a contiguous region of the array, reducing cache-line contention between threads.
- **PCC**: Each thread reads the full mean-subtracted matrix rows it needs; rows are accessed sequentially, maintaining good cache behaviour.

5.3 Load Balancing

- **Odd-Even Sort**: The per-thread workload is perfectly balanced since each active phase has the same number of pairs and these are distributed evenly.
- **PCC**: The interleaved (round-robin) row assignment in step 3 distributes the triangular workload evenly across threads, avoiding the imbalance that would arise from a simple contiguous partition.

6 Conclusion

Both odd-even sort and the Pearson correlation coefficient were parallelised successfully using C++ `std::thread` for shared-memory execution. Correctness was validated against the provided sequential reference programs.